

ENGI-9874-001 GR_37

Project Members

Last Name, First Name

- Fang, Zhou
- Patel, Vivek Thakorbhai
- Patel, Kevin Harshadkumar

Library Management System

- A simple system used to practice software design/specification and design patterns
- Patterns used (8)
 - decorator
 - strategy
 - command
 - observer
 - singleton
 - factory
 - abstract factory
 - adapter

How to use the system

Step 1

- Install required packages
 - tkinter (tk)
 - abc (is already a built-in std package)

Step 2

- Run the notebook and the program will show up
 - Add book with any name of book/author
 - factory pattern
 - observer pattern notifier
 - Add 3rd party books
 - adapter pattern hard-coded books
 - Add member
 - abstract factory pattern
 - observer pattern observer
 - Search the book you added with the names
 - strategy pattern
 - Borrow the book with given name
 - command pattern
 - Add accessory when borrowing a book
 - decorator pattern
 - Return the book with given name
 - command pattern

Note. The library object in the program - Singleton pattern

```
In [ ]: %pip install tk
```

Requirement already satisfied: tk in c:\users\7950x3d\appdata\local\programs\python\python312\lib\site-packages (0.1.0)
Note: you may need to restart the kernel to use updated packages.

```
In [ ]: # Singleton Pattern
class Library:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super(Library, cls).__new__(cls)
            cls._instance.books = []
            cls._instance.borrowed_books = []
            cls._instance.members = []
        return cls._instance
```

```

def borrow_book(self, book):
    if book in self.books:
        self.books.remove(book)
        self.borrowed_books.append(book)
        return True
    return False

def return_book(self, book):
    if book in self.borrowed_books:
        self.borrowed_books.remove(book)
        self.books.append(book)
        return True
    return False

```

```
In [ ]: from abc import ABC, abstractmethod
```

```

# Factory Method Pattern
class BookFactory:
    @staticmethod
    def create_book(book_type, title, author, book=None):
        if book_type == "EBook":
            return EBook(title, author)
        elif book_type == "PrintBook":
            return PrintBook(title, author)
        else:
            return None

```

```
In [ ]: # Abstract Factory Pattern
```

```

class MemberFactory(ABC):
    @abstractmethod
    def create_member(self, name):
        pass

class StudentMemberFactory(MemberFactory):
    def create_member(self, name):
        return StudentMember(name)

class FacultyMemberFactory(MemberFactory):
    def create_member(self, name):
        return FacultyMember(name)

```

```
In [ ]: # Decorator Pattern
```

```

class Book:
    def __init__(self, title: str, author: str):
        self.title = title
        self.author = author

    def get_description(self):
        return f"{self.title} by {self.author}"

class BookAccessoryDecorator(Book):
    def __init__(self, name, book_or_accessory: Book):
        self._name = name
        self._book_or_accessory = book_or_accessory

    def get_description(self):
        return f"{self._book_or_accessory.get_description()}\nAssesory:{self._name}"

class EBook(Book):
    pass

class PrintBook(Book):
    pass

```

```
In [ ]: # test decorator pattern
```

```

a = BookAccessoryDecorator("pen", Book("book1", "author1"))
b = BookAccessoryDecorator("pencil", a)
c = BookAccessoryDecorator("erasor", b)
print(c.get_description())

```

```

book1 by author1
Assesory:pen
Assesory:pencil
Assesory:erasor

```

```
In [ ]: # Observer Pattern
```

```

class Notifier:
    def __init__(self):
        self._observers = []
        self.new_book = None

    def attach(self, observer):
        self._observers.append(observer)

    def detach(self, observer):
        self._observers.remove(observer)

```

```

    def notify(self):
        for observer in self._observers:
            observer.update(self)

class LibraryNotifier(Notifier):
    def __init__(self):
        super().__init__()

    def add_new_book(self, book):
        self.new_book = book
        self.notify()

class Member(ABC):
    @abstractmethod
    def update(self, notifier):
        pass

class StudentMember(Member):
    def __init__(self, name):
        self.name = name

    def update(self, notifier):
        print(f"Student {self.name} notified of new book: {notifier.new_book.get_description()}")

class FacultyMember(Member):
    def __init__(self, name):
        self.name = name

    def update(self, notifier):
        print(f"Faculty {self.name} notified of new book: {notifier.new_book.get_description()}")

```

```

In [ ]: # Strategy Pattern
class SearchStrategy(ABC):
    @abstractmethod
    def search(self, library, query):
        pass

class TitleSearchStrategy(SearchStrategy):
    def search(self, library, query):
        available_books = [book for book in library.books if query.lower() in book.title.lower()]
        borrowed_books = [book for book in library.borrowed_books if query.lower() in book.title.lower()]
        return available_books, borrowed_books

class AuthorSearchStrategy(SearchStrategy):
    def search(self, library, query):
        available_books = [book for book in library.books if query.lower() in book.author.lower()]
        borrowed_books = [book for book in library.borrowed_books if query.lower() in book.author.lower()]
        return available_books, borrowed_books

```

```

In [ ]: # Command Pattern
class Command(ABC):
    @abstractmethod
    def execute(self, title):
        pass
    @abstractmethod
    def clearResult(self):
        pass

class BorrowBookCommand(Command):
    def __init__(self, library):
        self.library = library
        self.result = ""

    def execute(self, title):
        book = next((book for book in self.library.books if book.title == title), None)
        if book and self.library.borrow_book(book):
            self.result = f"Borrowed book: {book.get_description()}"
        else:
            self.result = f"Book '{title}' not found or already borrowed"

    def clearResult(self):
        self.result = ""

class ReturnBookCommand(Command):
    def __init__(self, library):
        self.library = library
        self.result = ""

    def execute(self, title):
        book = next((book for book in self.library.borrowed_books if book.title == title), None)
        if book and self.library.return_book(book):
            self.result = f"Returned book: {book.get_description()}"
        else:
            self.result = f"Book '{title}' not found in borrowed books"

```

```
def clearResult(self):
    self.result = ""
```

```
In [ ]: # Adapter Pattern
class ThirdPartyBookAPI:
    """As lagecy books that need to be adapted"""
    def get_books(self):
        return [{"title": "ThirdPartyBook1", "author": "Author1"}, {"title": "ThirdPartyBook2", "author": "Author2"}]

class BookAdapter(Book):
    def __init__(self, third_party_book):
        super().__init__(third_party_book["title"], third_party_book["author"])
```

```
In [ ]: import tkinter as tk
from tkinter import messagebox

# Config
SUPPORTED_BOOK_TYPES = ["EBook", "PrintBook"]
SEARCH_STRATEGIES = {
    "Search by Title": TitleSearchStrategy,
    "Search by Author": AuthorSearchStrategy
}
PREDEFINED_MEMBERS = [
    ("Alice", "Student"),
    ("Dr. Smith", "Faculty")
]

# Client Class using Tkinter
class Client(tk.Tk):
    def __init__(self):
        super().__init__()

    ### Class Diagram Critical Info ###
    self.search_strategies: dict[str: SearchStrategy] = SEARCH_STRATEGIES # 1. strategy pattern object - @Client.search_strategies
    self.library : 'Library' = Library() # 2. singleton pattern object
    self.notifier: 'Notifier' = LibraryNotifier() # 3. observer pattern object - @Client.add_notifier
    self.borrow_command: 'Command' = BorrowBookCommand(self.library) # 4. command pattern object - @Client.borrow_book
    self.return_command: 'Command' = ReturnBookCommand(self.library) # 5. command pattern object - @Client.return_book
    self.student_factory: 'MemberFactory' = StudentMemberFactory() # 5. abstract factory pattern object
    self.faculty_factory: 'MemberFactory' = FacultyMemberFactory() # 6. abstract factory pattern object - @Client.add_faculty
    self.bookfactory: 'BookFactory' = BookFactory # 6. factory pattern object - @Client.add_book
    # 7. adpater pattern: @Client.add_third_party_books, @Class.ThirdPartyBookAPI, @Class.BookAdapter
    # 8. decorator pattern: @Class.BookAccessory, @Client.prompt_for_accessories, @Client.borrow_book
    ### Class Diagram Info End ###

    # Title
    self.title("Library Management System")
    self.geometry("")
    self.create_widgets()
    # Configure grid layout to expand with window size
    self.grid_columnconfigure(1, weight=1)
    self.grid_columnconfigure(2, weight=1)
    self.grid_columnconfigure(3, weight=1)
    self.grid_columnconfigure(4, weight=1)
    self.grid_columnconfigure(5, weight=1)
    self.grid_rowconfigure(0, weight=1)
    self.grid_rowconfigure(1, weight=1)
    self.grid_rowconfigure(2, weight=1)

    # Initialize and register members
    self.init_members()

    def init_members(self):
        for name, member_type in PREDEFINED_MEMBERS:
            if member_type == "Student":
                member = self.student_factory.create_member(name)
            elif member_type == "Faculty":
                member = self.faculty_factory.create_member(name)
            self.library.members.append(member) # add memberto Library
            self.notifier.attach(member) # add observer to notifier

    def create_widgets(self):
        # Add book section
        tk.Label(self, text="Add Book").grid(row=0, column=0, padx=10, pady=10, sticky="e")

        # Dynamic selector for book type
        self.book_type_var = tk.StringVar(value=SUPPORTED_BOOK_TYPES[0])
        book_type_selector = tk.OptionMenu(self, self.book_type_var, *SUPPORTED_BOOK_TYPES)
        book_type_selector.grid(row=0, column=1, padx=10, pady=10, sticky="ew")

        self.title_var = tk.StringVar()
        self.author_var = tk.StringVar()

        tk.Label(self, text="Title:").grid(row=0, column=2, padx=10, pady=10, sticky="e")
        tk.Entry(self, textvariable=self.title_var).grid(row=0, column=3, padx=10, pady=10, sticky="ew")
```

```

tk.Label(self, text="Author:").grid(row=0, column=4, padx=10, pady=10, sticky="e")
tk.Entry(self, textvariable=self.author_var).grid(row=0, column=5, padx=10, pady=10, sticky="ew")

tk.Button(self, text="Add", command=self.add_book).grid(row=0, column=6, padx=10, pady=10, sticky="ew")

# Button to add third-party books
tk.Button(self, text="Add Third-Party Books", command=self.add_third_party_books).grid(row=0, column=7, padx=10, pady=

# Search book section
tk.Label(self, text="Search Book").grid(row=1, column=0, padx=10, pady=10, sticky="e")
self.search_var = tk.StringVar()
self.search_result_var = tk.StringVar()

tk.Label(self, text="Query:").grid(row=1, column=1, padx=10, pady=10, sticky="e")
tk.Entry(self, textvariable=self.search_var).grid(row=1, column=2, padx=10, pady=10, sticky="ew")

# Dynamically create search buttons based on available strategies
col = 3
for strategy_name, strategy_class in self.search_strategies.items():
    tk.Button(self, text=strategy_name, command=lambda s=strategy_class: self.search_books(s)).grid(row=1, column=col,
    col += 1

tk.Label(self, textvariable=self.search_result_var).grid(row=1, column=col, padx=10, pady=10, sticky="ew")

# Borrow and Return book section
tk.Label(self, text="Borrow/Return Book").grid(row=2, column=0, padx=10, pady=10, sticky="e")
self.borrow_return_var = tk.StringVar()

tk.Label(self, text="Book Title:").grid(row=2, column=1, padx=10, pady=10, sticky="e")
tk.Entry(self, textvariable=self.borrow_return_var).grid(row=2, column=2, padx=10, pady=10, sticky="ew")
tk.Button(self, text="Borrow", command=self.borrow_book).grid(row=2, column=3, padx=10, pady=10, sticky="ew")
tk.Button(self, text="Return", command=self.return_book).grid(row=2, column=4, padx=10, pady=10, sticky="ew")

# Add member section
tk.Label(self, text="Add Member").grid(row=3, column=0, padx=10, pady=10, sticky="e")

self.member_name_var = tk.StringVar()
self.member_type_var = tk.StringVar(value="Student")

tk.Label(self, text="Name:").grid(row=3, column=1, padx=10, pady=10, sticky="e")
tk.Entry(self, textvariable=self.member_name_var).grid(row=3, column=2, padx=10, pady=10, sticky="ew")

member_type_selector = tk.OptionMenu(self, self.member_type_var, "Student", "Faculty")
member_type_selector.grid(row=3, column=3, padx=10, pady=10, sticky="ew")

tk.Button(self, text="Add Member", command=self.add_member).grid(row=3, column=4, padx=10, pady=10, sticky="ew")

def add_member(self):
    name = self.member_name_var.get()
    member_type = self.member_type_var.get()

    if member_type == "Student":
        member = self.student_factory.create_member(name)
    elif member_type == "Faculty":
        member = self.faculty_factory.create_member(name)

    self.library.members.append(member)      # add member to library
    self.notifier.attach(member)             # add observer to notifier
    messagebox.showinfo("Success", f"Added {member_type} member: {name}")

def add_book(self):
    book_type = self.book_type_var.get()
    title = self.title_var.get()
    author = self.author_var.get()
    book = self.bookfactory.create_book(book_type, title, author)
    if book:
        self.library.books.append(book)
        self.notifier.add_new_book(book)
        messagebox.showinfo("Success", f"Added {book.get_description()}")
    else:
        messagebox.showerror("Error", "Failed to add book")

def search_books(self, strategy):
    query = self.search_var.get()
    available_books, borrowed_books = strategy().search(self.library, query)
    self.display_search_results(available_books, borrowed_books)

def display_search_results(self, available_books, borrowed_books):
    if available_books or borrowed_books:
        result_text = ""
        if available_books:
            result_text += "Available books:\n"
            result_text += "\n".join([book.get_description() for book in available_books]) + "\n"
        if borrowed_books:
            result_text += "Borrowed books:\n"
            result_text += "\n".join([book.get_description() for book in borrowed_books])

```

```

        self.search_result_var.set(result_text)
    else:
        self.search_result_var.set("No results found")
messagebox.showinfo("Search Results", self.search_result_var.get())
def borrow_book(self):
    title = self.borrow_return_var.get()
    self.borrow_command.execute(title)

    if "Borrowed book:" in self.borrow_command.result:
        borrowed_book = next((book for book in self.library.borrowed_books if book.title == title), None)
        if borrowed_book:
            borrowed_book_with_accessories = self.prompt_for_accessories(borrowed_book)
            messagebox.showinfo("Result", f"Book borrowed with accessories: {borrowed_book_with_accessories.get_description()}")
        else:
            messagebox.showerror("Error", "Book not found after borrowing")
    else:
        messagebox.showinfo("Result", self.borrow_command.result)

    self.borrow_command.clearResult()

def return_book(self):
    title = self.borrow_return_var.get()
    self.return_command.execute(title)
    messagebox.showinfo("Result", self.return_command.result)
    self.return_command.clearResult()

def add_third_party_books(self):
    third_party_api = ThirdPartyBookAPI()
    third_party_books = third_party_api.get_books()
    for third_party_book in third_party_books:
        adapted_book = BookAdapter(third_party_book)
        self.library.books.append(adapted_book)
        self.notifier.add_new_book(adapted_book)
    messagebox.showinfo("Success", "Third-party books added to the library.")

def prompt_for_accessories(self, book):
    accessory_window = tk.Toplevel(self)
    accessory_window.title("Add Accessories")

    tk.Label(accessory_window, text="Enter accessory:").grid(row=0, column=0, padx=10, pady=10)
    accessory_var = tk.StringVar()
    tk.Entry(accessory_window, textvariable=accessory_var).grid(row=0, column=1, padx=10, pady=10)

    def add_accessory():
        accessory = accessory_var.get()
        if accessory:
            nonlocal book
            book = BookAccessoryDecorator(accessory, book)
            accessory_var.set("") # Clear the entry field

    def done():
        accessory_window.destroy()

    tk.Button(accessory_window, text="Add Accessory", command=add_accessory).grid(row=1, column=0, padx=10, pady=10)
    tk.Button(accessory_window, text="Done", command=done).grid(row=1, column=1, padx=10, pady=10)

    accessory_window.transient(self)
    accessory_window.grab_set()
    self.wait_window(accessory_window)

    return book

```

```

In [ ]: client = Client()
        client.mainloop()

```

```

Student Alice notified of new book: new book by sample author
Faculty Dr. Smith notified of new book: new book by sample author
Student Alice notified of new book: ThirdPartyBook1 by Author1
Faculty Dr. Smith notified of new book: ThirdPartyBook1 by Author1
Student Alice notified of new book: ThirdPartyBook2 by Author2
Faculty Dr. Smith notified of new book: ThirdPartyBook2 by Author2

```